

Matching Regular-Typed Pattern Languages: Quadratic-Time Algorithms

Yuya Uezato

CyberAgent, Inc., Japan

CPM 2026

Our Problem: **Membership Problem**

Deciding whether an input text W belongs to a language:

$$W \in L_C(P) ?$$

Our Problem: Membership Problem

Deciding whether an input text W belongs to a language:

$$\overset{\text{input text}}{\overline{W}} \in L_{\underset{\text{regular type}}{C}}\left(\overset{\text{input pattern}}{\overline{P}}\right) ?$$

Main object: regular-typed pattern expressions $\mathcal{R} = (C, P)$;
and its language $L_C(P)$.

Our Problem: Membership Problem

Deciding whether an input text W belongs to a language:

$$\overbrace{W}^{\text{input text}} \in L_{\underbrace{C}_{\text{regular type}}}(\overbrace{P}^{\text{input pattern}}) ?$$

Main object: regular-typed pattern expressions $\mathcal{R} = (C, P)$;
and its language $L_C(P)$.

Main Result

For regular-typed patterns with one repeated variable, the membership problem can be solved in

$$O(|W|^2 \text{poly}(|\mathcal{R}|)) \text{ time.}$$

Prelim.: Pattern Expressions *without Types*

A pattern P is a string over alphabet symbols and variables.

Let $\Sigma = \{a, b, \#\}$ be an alphabet and let x be a variable.

$$\text{(Example) } P = x \# \# x .$$

The two $\#$'s in P are literal alphabet symbols.

Prelim.: Pattern Expressions *without Types*

A pattern P is a string over alphabet symbols and variables.

Let $\Sigma = \{a, b, \#\}$ be an alphabet and let x be a variable.

$$\text{(Example) } P = x \# \# x .$$

The two $\#$'s in P are literal alphabet symbols.

The language of P is generated by assigning an arbitrary string $w \in \Sigma^*$ to x :

$$L(P) := \{ w_x \# \# w_x : w_x \in \Sigma^* \}.$$

Prelim.: Pattern Expressions *without Types*

A pattern P is a string over alphabet symbols and variables.

Let $\Sigma = \{a, b, \#\}$ be an alphabet and let x be a variable.

$$\text{(Example) } P = x \# \# x .$$

The two $\#$'s in P are literal alphabet symbols.

The language of P is generated by assigning an arbitrary string $w \in \Sigma^*$ to x :

$$L(P) := \{ w_x \# \# w_x : w_x \in \Sigma^* \}.$$

Membership Example

$$\text{aba} \# \# \text{aba} \in L(P), \quad \text{by } x \mapsto \text{aba},$$

Prelim.: Pattern Expressions *without Types*

A pattern P is a string over alphabet symbols and variables.

Let $\Sigma = \{a, b, \#\}$ be an alphabet and let x be a variable.

$$\text{(Example) } P = x \# \# x .$$

The two $\#$'s in P are literal alphabet symbols.

The language of P is generated by assigning an arbitrary string $w \in \Sigma^*$ to x :

$$L(P) := \{ w_x \# \# w_x : w_x \in \Sigma^* \}.$$

Membership Example

$aba \# \# aba \in L(P),$

by $x \mapsto aba,$

$aba \# \# babb \notin L(P),$

because $aba \neq baab$

Prelim.: Pattern Expressions *without Types*

A pattern P is a string over alphabet symbols and variables.

Let $\Sigma = \{a, b, \#\}$ be an alphabet and let x be a variable.

$$\text{(Example) } P = x \# \# x .$$

The two $\#$'s in P are literal alphabet symbols.

The language of P is generated by assigning an arbitrary string $w \in \Sigma^*$ to x :

$$L(P) := \{ w_x \# \# w_x : w_x \in \Sigma^* \}.$$

Membership Example

$aba \# \# aba \in L(P),$	by $x \mapsto aba,$
$aba \# \# babb \notin L(P),$	because $aba \neq baab$
$a \# \# b \# \# a \# \# b \in L(P),$	by $x \mapsto a \# \# b.$

Main Object: Pattern Expressions with *Regular Types*

For $P = x \#\# x$, the variable x can be instantiated with any string $\in \Sigma^*$: i.e.,

$$a \#\# b \#\# a \#\# b \in L(P) \quad \text{by} \quad x \mapsto a \#\# b.$$

Main Object: Pattern Expressions with *Regular Types*

For $P = x \#\# x$, the variable x can be instantiated with any string $\in \Sigma^*$: i.e.,

$$a \#\# b \#\# a \#\# b \in L(P) \quad \text{by} \quad x \mapsto a \#\# b.$$

Motivation of Type:

We would like to restrict the *domain* of strings assigned for x .

Main Object: Pattern Expressions with *Regular Types*

For $P = x \#\# x$, the variable x can be instantiated with any string $\in \Sigma^*$: i.e.,

$$a \#\# b \#\# a \#\# b \in L(P) \quad \text{by} \quad x \mapsto a \#\# b.$$

Motivation of Type:

We would like to restrict the *domain* of strings assigned for x .

Regular type = Domain Restriction by Regular Expressions

$$\text{(Example): } C(x) = (\Sigma \setminus \{\#\})^* = \{a, b\}^*.$$

Main Object: Pattern Expressions with *Regular Types*

For $P = x \#\# x$, the variable x can be instantiated with any string $\in \Sigma^*$: i.e.,

$$a \#\# b \#\# a \#\# b \in L(P) \quad \text{by} \quad x \mapsto a \#\# b.$$

Motivation of Type:

We would like to restrict the *domain* of strings assigned for x .

Regular type = Domain Restriction by Regular Expressions

$$\text{(Example): } C(x) = (\Sigma \setminus \{\#\})^* = \{a, b\}^*.$$

Then, the regular-typed pattern language $L_C(P)$ is defined as follows:

$$L_C(P) = \{ w_x \#\# w_x : w_x \in C(x) \} = \{ w_x \#\# w_x : w_x \in \{a, b\}^* \}.$$

Main Object: Pattern Expressions with *Regular Types*

For $P = x \#\# x$, the variable x can be instantiated with any string $\in \Sigma^*$: i.e.,

$$a \#\# b \#\# a \#\# b \in L(P) \quad \text{by} \quad x \mapsto a \#\# b.$$

Motivation of Type:

We would like to restrict the *domain* of strings assigned for x .

Regular type = Domain Restriction by Regular Expressions

$$\text{(Example): } C(x) = (\Sigma \setminus \{\#\})^* = \{a, b\}^*.$$

Then, the regular-typed pattern language $L_C(P)$ is defined as follows:

$$L_C(P) = \{ w_x \#\# w_x : w_x \in C(x) \} = \{ w_x \#\# w_x : w_x \in \{a, b\}^* \}.$$

Effect of the type

$$aba \#\# aba \in L_C(P), \quad a \#\# b \#\# a \#\# b \notin L_C(P).$$

Example ①: Substring Search via Regular-typed Patterns

Let $\Sigma = \{a, b, c, \dots, z, \#\}$ and let α, γ, β be variables. Consider the pattern

$$P = \beta \# \alpha \beta \gamma.$$

Example ①: Substring Search via Regular-typed Patterns

Let $\Sigma = \{a, b, c, \dots, z, \#\}$ and let α, γ, β be variables. Consider the pattern

$$P = \beta \# \alpha \beta \gamma.$$

Assign regular types that forbid the separator inside variables:

$$\text{WORD} = C(\alpha) = C(\gamma) = C(\beta) = (\Sigma \setminus \{\#\})^*.$$

Example ①: Substring Search via Regular-typed Patterns

Let $\Sigma = \{a, b, c, \dots, z, \#\}$ and let α, γ, β be variables. Consider the pattern

$$P = \beta \# \alpha \beta \gamma.$$

Assign regular types that forbid the separator inside variables:

$$\text{WORD} = C(\alpha) = C(\gamma) = C(\beta) = (\Sigma \setminus \{\#\})^*.$$

$$W \in L_C(P) \iff W = \overbrace{w_\beta}^{\in \text{WORD}} \# \overbrace{w_\alpha}^{\in \text{WORD}} \overbrace{w_\beta}^{\in \text{WORD}} \overbrace{w_\gamma}^{\in \text{WORD}}$$

Example ①: Substring Search via Regular-typed Patterns

Let $\Sigma = \{a, b, c, \dots, z, \#\}$ and let α, γ, β be variables. Consider the pattern

$$P = \beta \# \alpha \beta \gamma.$$

Assign regular types that forbid the separator inside variables:

$$\text{WORD} = C(\alpha) = C(\gamma) = C(\beta) = (\Sigma \setminus \{\#\})^*.$$

$$W \in L_C(P) \iff W = \overbrace{w_\beta}^{\in \text{WORD}} \# \overbrace{w_\alpha}^{\in \text{WORD}} \overbrace{w_\beta}^{\in \text{WORD}} \overbrace{w_\gamma}^{\in \text{WORD}}$$

Membership examples

$\text{aba} \# \text{c aba d} \in L_C(P), \quad \beta \mapsto \text{aba}, \alpha \mapsto \text{c}, \gamma \mapsto \text{d},$

$\text{aba} \# \text{c abz d} \notin L_C(P).$

Example ②: Repeated Delimiters

Markdown-style:
Matching the number of backquotes

$P = \alpha_1 \beta \alpha_2 \beta \alpha_3$, $C(\beta) := `^+ = \text{backquote}^+$.

TEXT

TEXT

TEXT

TEXT

TEXT

Example ②: Repeated Delimiters

Markdown-style:
Matching the number of backquotes

$P = \alpha_1 \beta \alpha_2 \beta \alpha_3$, $C(\beta) := `^+ = \text{backquote}^+$.

```
TEXT
~~~~~
TEXT
~~~~~
TEXT
~~~~~
TEXT
~~~~~
TEXT
```

} α_2

Ruby (Programming Language):
Heredocument / String delimiters

$P = \beta \alpha \beta$, $C(\beta) = \text{identifier-like strings}$.

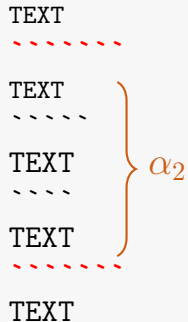
```
string_var = <<ArbitraryDelim
TEXT
hogefugapiyo
TEXT
ArbitraryDelim

print(string_var)
```

Example ②: Repeated Delimiters

Markdown-style:
Matching the number of backquotes

$P = \alpha_1 \beta \alpha_2 \beta \alpha_3$, $C(\beta) := `^+ = \text{backquote}^+$.



The diagram shows a sequence of text and backquote delimiters. It starts with 'TEXT' followed by a red dashed line representing a backquote. Then, 'TEXT' is followed by four backquotes. Next, 'TEXT' is followed by three backquotes. Then, 'TEXT' is followed by two backquotes. Finally, 'TEXT' is followed by a red dashed line representing a backquote. A large orange curly brace on the right groups the four backquote sequences (the four backquotes, the three backquotes, the two backquotes, and the final backquote) and is labeled α_2 .

Ruby (Programming Language):
Heredocument / String delimiters

$P = \beta \alpha \beta$, $C(\beta) = \text{identifier-like strings}$.

```
string_var = <<ArbitraryDelim
TEXT
hogefugapiyo
TEXT
ArbitraryDelim

print(string_var)
```

A repeated variable stores a delimiter and requires the same one later.

Formal Definition of Our Problem: Single Repeated Variable

This talk and the paper focus on regular-typed patterns of the following form:

$$P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K \quad (K \geq 3).$$

Formal Definition of Our Problem: Single Repeated Variable

This talk and the paper focus on regular-typed patterns of the following form:

$$P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K \quad (K \geq 3).$$

Our Examples

$$P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3,$$

$$P_4 = \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

Formal Definition of Our Problem: Single Repeated Variable

This talk and the paper focus on regular-typed patterns of the following form:

$$P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K \quad (K \geq 3).$$

Our Examples

$$P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3,$$

$$P_4 = \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

~~Examples Outside Our Scope~~

~~$$\alpha @ \beta \# \beta @ \alpha,$$~~

~~$$\alpha\alpha\beta\beta, \alpha\alpha\beta\beta\gamma\gamma.$$~~

Formal Definition of Our Problem: Single Repeated Variable

This talk and the paper focus on regular-typed patterns of the following form:

$$P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K \quad (K \geq 3).$$

Our Examples

$$P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3,$$

$$P_4 = \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

~~Examples Outside Our Scope~~

~~$$\alpha @ \beta \# \beta @ \alpha,$$~~

~~$$\alpha\alpha\beta\beta, \alpha\alpha\beta\beta\gamma\gamma.$$~~

Our Task: Membership Problem

Input 1: a regular-typed pattern expression $\mathcal{R} = (P_K, C)$

Input 2: a string W

Formal Definition of Our Problem: Single Repeated Variable

This talk and the paper focus on regular-typed patterns of the following form:

$$P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K \quad (K \geq 3).$$

Our Examples

$$P_3 = \alpha_1 \beta \alpha_2 \beta \alpha_3,$$

$$P_4 = \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

~~Examples Outside Our Scope~~

~~$$\alpha @ \beta \# \beta @ \alpha,$$~~

~~$$\alpha\alpha\beta\beta, \alpha\alpha\beta\beta\gamma\gamma.$$~~

Our Task: Membership Problem

Input 1: a regular-typed pattern expression $\mathcal{R} = (P_K, C)$

Input 2: a string W

Task: Find a decomposition such as

$$(K = 3) \quad W = \begin{array}{c} \overbrace{\quad \quad}^{\in \mathcal{A}_1} \quad \overbrace{\quad}^{\in \mathcal{B}} \quad \overbrace{\quad \quad}^{\in \mathcal{A}_2} \quad \overbrace{\quad}^{\in \mathcal{B}} \quad \overbrace{\quad \quad}^{\in \mathcal{A}_3} \\ \boxed{w_1} \quad \boxed{u} \quad \boxed{w_2} \quad \boxed{u} \quad \boxed{w_3} \end{array}$$

Why Consider Single Repeated Variable Form $\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$?

Related work viewpoint

Work	Generality	Type	Complexity
Fernau et al.	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	NO	$O(W ^2 \text{poly}(P))$
Nogami–Terauchi	Only $K = 3$: $\alpha_1 \beta \alpha_2 \beta \alpha_3$	YES	$O(W ^2 \text{poly}(P + C))$
This work	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	YES	$O(W ^2 \text{poly}(P + C))$

Why Consider Single Repeated Variable Form $\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$?

Related work viewpoint

Work	Generality	Type	Complexity
Fernau et al.	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	NO	$O(W ^2 \text{poly}(P))$
Nogami–Terauchi	Only $K = 3$: $\alpha_1 \beta \alpha_2 \beta \alpha_3$	YES	$O(W ^2 \text{poly}(P + C))$
This work	$\alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$	YES	$O(W ^2 \text{poly}(P + C))$

As Regular Expression with *Backreferences*

Single repeated variable form is one of the base case of regular expression with *backreferences*.
For example, $K = 3$,

$$\alpha_1 \beta \alpha_2 \beta \alpha_3 \approx A_1 (B)_1 A_2 \setminus 1 A_3,$$

where $A_i = C(\alpha_i)$ and $B = C(\beta)$.

So the form is restricted, but it is a standard and meaningful base case.

Talk Overview

So far

- We introduced regular-typed pattern expressions. ✓
- We saw examples: substring search and repeated delimiters. ✓
- We fixed the target form: $P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$. ✓

Talk Overview

So far

- We introduced regular-typed pattern expressions. ✓
- We saw examples: substring search and repeated delimiters. ✓
- We fixed the target form: $P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K$. ✓

From now on: algorithm

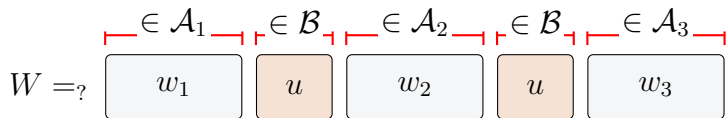
I will explain the algorithm by focusing on the smallest nontrivial cases:

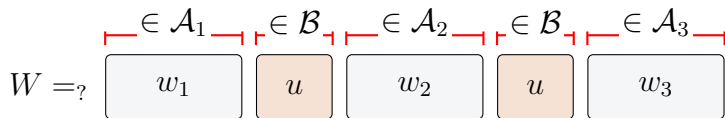
$$K = 3 : \quad \alpha_1 \beta \alpha_2 \beta \alpha_3,$$

then

$$K = 4 : \quad \alpha_1 \beta \alpha_2 \beta \alpha_3 \beta \alpha_4.$$

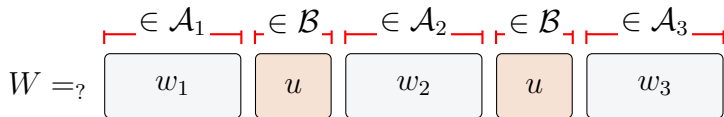
$K = 3$ is the key case; general $K \geq 4$ is a simple extension.





Algorithm

Build Data Structure for DFA Queries and Subs_{occ}(w) in $O(|W|^2)$ time.

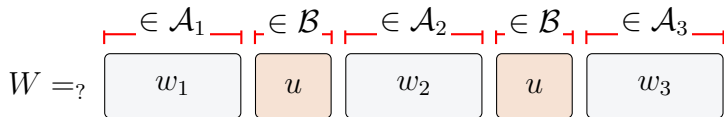


Algorithm

Build Data Structure for DFA Queries and Subs_{occ}(w) in $O(|W|^2)$ time.

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

▷ u is a substring of W and S_u is the sorted occurrences list of u



Algorithm

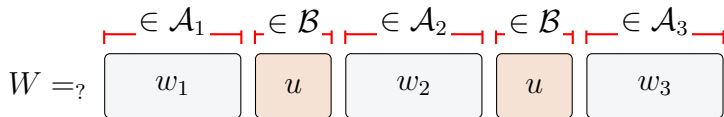
Build Data Structure for DFA Queries and Subs_{occ}(w) in $O(|W|^2)$ time.

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

 ▷ u is a substring of W and S_u is the sorted occurrences list of u

if $u \notin L(\mathcal{B})$ **then**

continue



Algorithm

Build Data Structure for DFA Queries and Subs_{occ}(w) in $O(|W|^2)$ time.

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

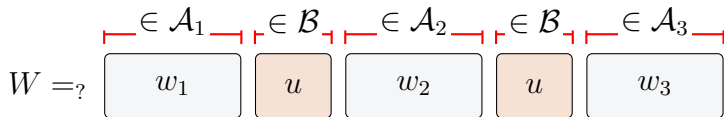
▷ u is a substring of W and S_u is the sorted occurrences list of u

if $u \notin L(\mathcal{B})$ **then**

└ **continue**

if $\text{Decompose}(u, S_u)$ **then** ▷ Check whether $w = w_1 u w_2 u w_3$ in $O(|S_u|)$ time

└ **return** ACCEPT



Algorithm

Build Data Structure for DFA Queries and Subs_{occ}(w) in $O(|W|^2)$ time.

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

▷ u is a substring of W and S_u is the sorted occurrences list of u

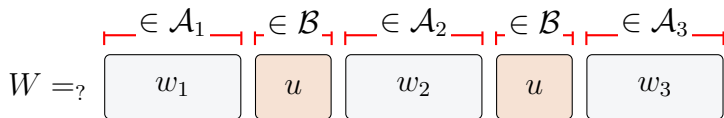
if $u \notin L(\mathcal{B})$ **then**

└ **continue**

if Decompose(u, S_u) **then** ▷ Check whether $w = w_1 u w_2 u w_3$ in $O(|S_u|)$ time

└ **return** ACCEPT

return REJECT



Algorithm

Build Data Structure for DFA Queries and Subs_{occ}(w) in $O(|W|^2)$ time.

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

▷ u is a substring of W and S_u is the sorted occurrences list of u

if $u \notin L(\mathcal{B})$ **then**

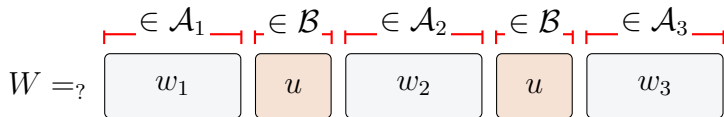
continue

if Decompose(u, S_u) **then** ▷ Check whether $w = w_1 u w_2 u w_3$ in $O(|S_u|)$ time

return ACCEPT

return REJECT

We observe $\sum_{\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)} |S_u| = O(|W|^2)$.



Algorithm

Build Data Structure for DFA Queries and Subs_{occ}(w) in $O(|W|^2)$ time.

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

▷ u is a substring of W and S_u is the sorted occurrences list of u

if $u \notin L(\mathcal{B})$ **then**

| **continue**

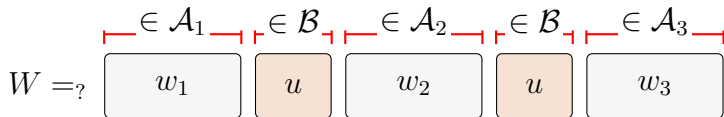
if Decompose(u, S_u) **then** ▷ Check whether $w = w_1 u w_2 u w_3$ in $O(|S_u|)$ time

| **return** ACCEPT

return REJECT

We observe $\sum_{\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)} |S_u| = O(|W|^2)$.

⇒ Our procedure runs in $O(|W|^2)$ time.



Algorithm

Build Data Structure for DFA Queries and Subs_{occ}(w) in $O(|W|^2)$ time.

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

▷ u is a substring of W and S_u is the sorted occurrences list of u

if $u \notin L(\mathcal{B})$ **then**

| **continue**

if Decompose(u, S_u) **then** ▷ Check whether $w = w_1 u w_2 u w_3$ in $O(|S_u|)$ time

| **return** ACCEPT

return REJECT

We observe $\sum_{\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)} |S_u| = O(|W|^2)$.

⇒ Our procedure runs in $O(|W|^2)$ time.

Remark: For the simplicity, we only consider the complexity on the input length $|W|$.

Algorithm

Build Data Structure for DFA Queries in $O(|W|^2)$ time



Build $\text{Subs}_{\text{occ}}(w)$ in $O(|W|^2)$ time

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

▷ $S_u = [p_1, p_2, \dots, p_r]$ is the sorted occurrences list of u

if $u \notin L(\mathcal{B})$ **then**

└ **continue**

if $\text{Decompose}(u, S_u)$ **then** ▷ *Check whether $w = w_1 u w_2 u w_3$ in $O(|S_u|)$ time*

└ **return** ACCEPT

return REJECT

Build Data Structures for DFA Queries

Notation: For a **deterministic** finite automaton (DFA) M and its state $q \in Q_M$, we write

$$q \xRightarrow{u} q'$$

to mean that, starting from q , reading u brings M to q' . Equivalently, $q' = \hat{\delta}_M(q, u)$.

Build Data Structures for DFA Queries

Notation: For a **deterministic** finite automaton (DFA) M and its state $q \in Q_M$, we write

$$q \xRightarrow{u} q'$$

to mean that, starting from q , reading u brings M to q' . Equivalently, $q' = \hat{\delta}_M(q, u)$.

For all q, i, j , we precompute $\Delta_M(q, i, j)$ totally in $O(|W|^2)$ time:

$$\Delta_M(q, i, j) := \hat{\delta}_M(q, W[i..j]).$$

Build Data Structures for DFA Queries

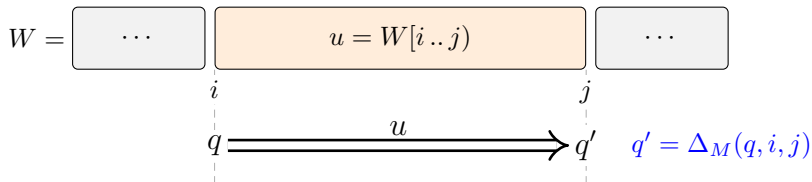
Notation: For a **deterministic** finite automaton (DFA) M and its state $q \in Q_M$, we write

$$q \xrightarrow{u} q'$$

to mean that, starting from q , reading u brings M to q' . Equivalently, $q' = \hat{\delta}_M(q, u)$.

For all q, i, j , we precompute $\Delta_M(q, i, j)$ totally in $O(|W|^2)$ time:

$$\Delta_M(q, i, j) := \hat{\delta}_M(q, W[i..j]).$$



Build Data Structures for DFA Queries

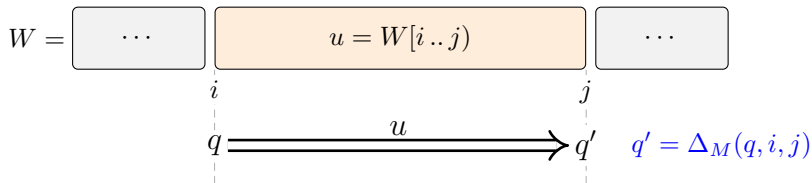
Notation: For a **deterministic** finite automaton (DFA) M and its state $q \in Q_M$, we write

$$q \xrightarrow{u} q'$$

to mean that, starting from q , reading u brings M to q' . Equivalently, $q' = \hat{\delta}_M(q, u)$.

For all q, i, j , we precompute $\Delta_M(q, i, j)$ totally in $O(|W|^2)$ time:

$$\Delta_M(q, i, j) := \hat{\delta}_M(q, W[i..j]).$$



Application: We can answer if M accepts the substring $W[i..j]$ in $O(1)$ time by checking that $\Delta_M(q_{\text{initial}}, i, j)$ is an accepting state.

Algorithm

Build Data Structure for DFA Queries in $O(|W|^2)$ time ✓

Build $\text{Subs}_{\text{occ}}(w)$ in $O(|W|^2)$ time

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

▷ $S_u = [p_1, p_2, \dots, p_r]$ is the sorted occurrences list of u

if $u \notin L(\mathcal{B})$ ✓ **then**

└ **continue**

if $\text{Decompose}(u, S_u)$ **then** ▷ Check whether $w = w_1 u w_2 u w_3$ in $O(|S_u|)$ time

└ **return** ACCEPT

return REJECT

Algorithm

Build Data Structure for DFA Queries in $O(|W|^2)$ time ✓

Build $\text{Subs}_{\text{occ}}(w)$ in $O(|W|^2)$ time 🧐

for all $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$ **do**

▷ $S_u = [p_1, p_2, \dots, p_r]$ is the sorted occurrences list of u

if $u \notin L(\mathcal{B})$ ✓ **then**

└ **continue**

if $\text{Decompose}(u, S_u)$ **then** ▷ Check whether $w = w_1 u w_2 u w_3$ in $O(|S_u|)$ time

└ **return** ACCEPT

return REJECT

Sub_{occ}(w): Computing all distinct substrings of w and their occurrences

For a nonempty substring u of W , define a sorted list of all starting positions of u in W :

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$

$$W = \left[\cdots \right] \boxed{u_1} \left[\cdots \right] \boxed{u_2} \left[\cdots \right] \boxed{u_3} \left[\cdots \right] \boxed{u_r} \left[\cdots \right]$$

$p_1 \qquad \qquad \qquad p_2 \qquad \qquad \qquad p_3 \qquad \qquad \qquad p_r$

Sub_{occ}(w): Computing all distinct substrings of w and their occurrences

For a nonempty substring u of W , define a sorted list of all starting positions of u in W :

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$

$$W = \begin{array}{ccccccc} \boxed{\dots} & \boxed{u_1} & \boxed{\dots} & \boxed{u_2} & \boxed{\dots} & \boxed{u_3} & \boxed{\dots\dots\dots} & \boxed{u_r} & \boxed{\dots} \\ & p_1 & & p_2 & & p_3 & & p_r & \end{array}$$

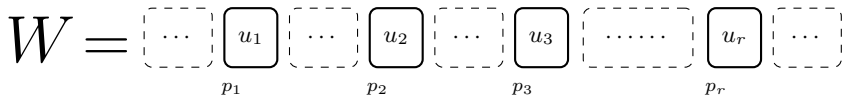
Example

$$W = \begin{array}{cccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ a & a & b & a & a & b & a & b & b & a \end{array},$$

Sub_{occ}(w): Computing all distinct substrings of w and their occurrences

For a nonempty substring u of W , define a sorted list of all starting positions of u in W :

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$



Example

1 2 3 4 5 6 7 8 9 10
 $W =$ a a b a a b a b b a ,

$$\text{Socc}(\text{ab}) = [2, 5, 7],$$

Sub_{occ}(w): Computing all distinct substrings of w and their occurrences

For a nonempty substring u of W , define a sorted list of all starting positions of u in W :

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$

$$W = \left[\cdots \right] \boxed{u_1} \left[\cdots \right] \boxed{u_2} \left[\cdots \right] \boxed{u_3} \left[\cdots \right] \boxed{u_r} \left[\cdots \right]$$

$p_1 \qquad \qquad p_2 \qquad \qquad p_3 \qquad \qquad p_r$

Example

$$W = \begin{array}{cccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ a & a & b & a & a & b & a & b & b & a \end{array},$$

$$\text{Socc}(ab) = [2, 5, 7], \quad \text{Socc}(ba) = [3, 6, 9].$$

Subs_{occ}(w): Computing all distinct substrings of w and their occurrences

For a nonempty substring u of W , define a sorted list of all starting positions of u in W :

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$

$$W = \begin{array}{ccccccc} \boxed{\dots} & \boxed{u_1} & \boxed{\dots} & \boxed{u_2} & \boxed{\dots} & \boxed{u_3} & \boxed{\dots\dots\dots} & \boxed{u_r} & \boxed{\dots} \\ & p_1 & & p_2 & & p_3 & & p_r & \end{array}$$

Example

$$W = \begin{array}{cccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ a & a & b & a & a & b & a & b & b & a \end{array},$$

$$\text{Socc}(ab) = [2, 5, 7], \quad \text{Socc}(ba) = [3, 6, 9].$$

Our useful data structure

$$\text{Subs}_{\text{occ}}(W) = \{ \langle u, \text{Socc}(u) \rangle : u \text{ is a nonempty substring of } W \}.$$

Subs_{occ}(w): Computing all distinct substrings of w and their occurrences

For a nonempty substring u of W , define a sorted list of all starting positions of u in W :

$$\text{Socc}(u) := [p_1, p_2, \dots, p_r] \quad (p_1 < p_2 < \dots < p_r) \text{ s.t. } W[p_i .. p_i + |u|) = u.$$

$$W = \begin{array}{ccccccc} \boxed{\dots} & \boxed{u_1} & \boxed{\dots} & \boxed{u_2} & \boxed{\dots} & \boxed{u_3} & \boxed{\dots\dots\dots} & \boxed{u_r} & \boxed{\dots} \\ & p_1 & & p_2 & & p_3 & & p_r & \end{array}$$

Example

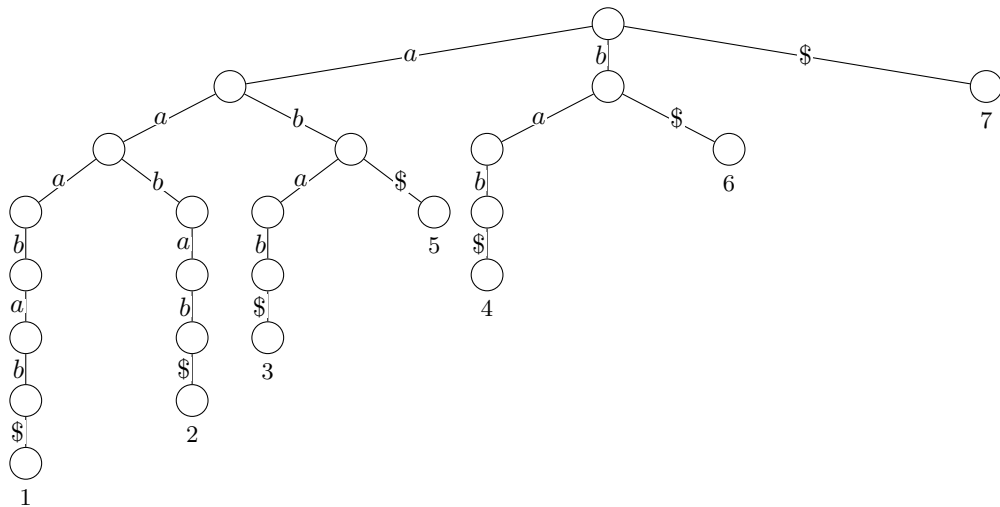
$$W = \begin{array}{cccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ a & a & b & a & a & b & a & b & b & a \end{array},$$

$$\text{Socc}(ab) = [2, 5, 7], \quad \text{Socc}(ba) = [3, 6, 9].$$

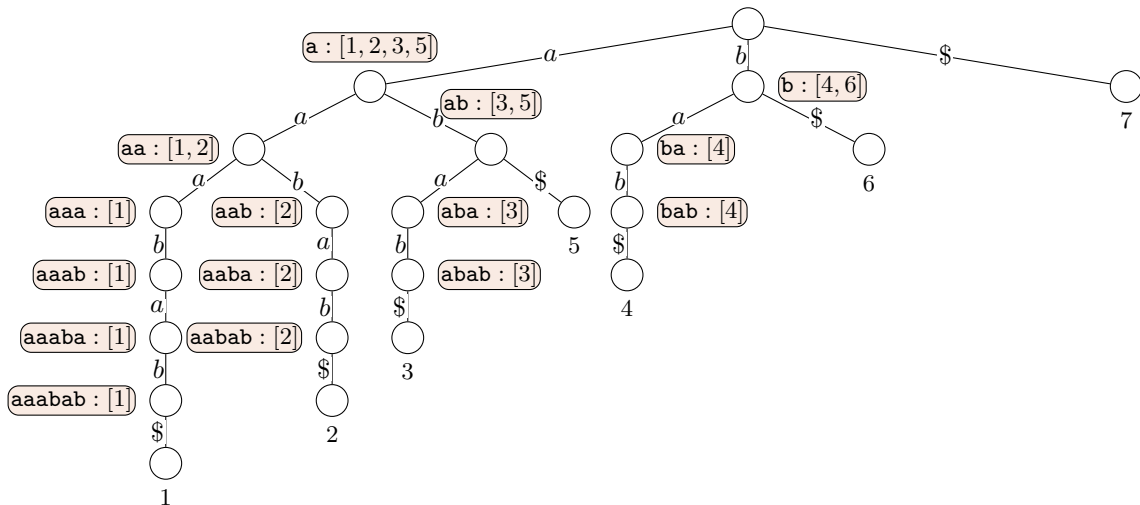
Our useful data structure

$$\text{Subs}_{\text{occ}}(W) = \{ \langle u, \text{Socc}(u) \rangle : u \text{ is a nonempty substring of } W \}.$$

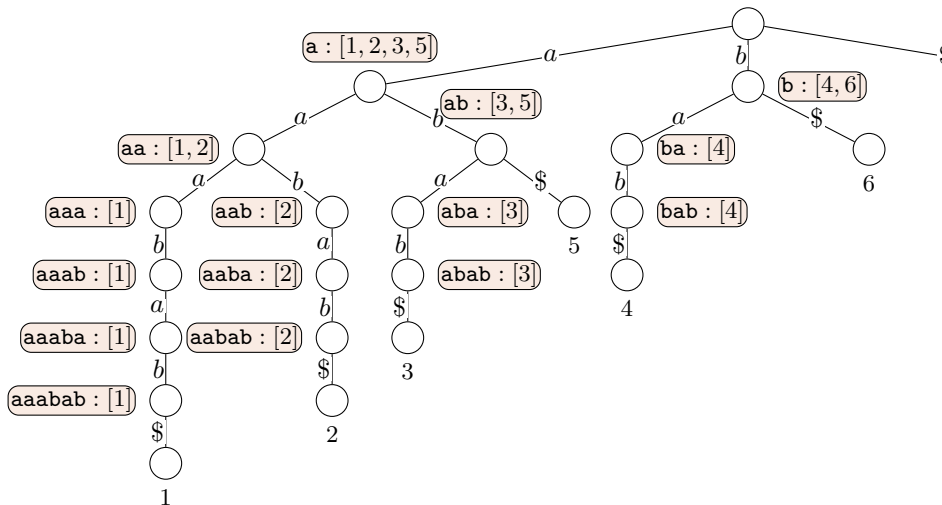
Example: Decorated Suffix Trie of $W = \text{aaabab}$ and $W\$ = \text{aaabab\$}$



Example: Decorated Suffix Trie of $W = \text{aaabab}$ and $W\$ = \text{aaabab\$}$



Example: Decorated Suffix Trie of $W = \text{aaabab}$ and $W\$ = \text{aaabab\$}$



Claim

$\text{Subs}_{\text{occ}}(W)$

can be built in

$O(|W|^2)$ time

by using
postorder
sorted-array
merging.

Note: Σ is fixed.

Algorithm

```
1 Build Data Structure for DFA Queries in  $O(|W|^2)$  time ✓
2 Build  $\text{Subs}_{\text{occ}}(w)$  in  $O(|W|^2)$  time ✓
3 for all  $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$  ✓ do
    ▷  $S_u = [p_1, p_2, \dots, p_r]$  is the sorted occurrences list of  $u$ 
4     if  $u \notin L(\mathcal{B})$  ✓ then
5         continue
6     if Decompose( $u, S_u$ ) then
7         ▷ Check whether  $w = w_1 u w_2 u w_3$  in  $O(|S_u|)$  time
8         return ACCEPT
9 return REJECT
```



Algorithm

```
1 Build Data Structure for DFA Queries in  $O(|W|^2)$  time ✓
2 Build  $\text{Subs}_{\text{occ}}(w)$  in  $O(|W|^2)$  time ✓
3 for all  $\langle u, S_u \rangle \in \text{Subs}_{\text{occ}}(W)$  ✓ do
    ▷  $S_u = [p_1, p_2, \dots, p_r]$  is the sorted occurrences list of  $u$ 
    4 if  $u \notin L(\mathcal{B})$  ✓ then
    5     continue
    6 if Decompose( $u, S_u$ )  then
    7     ▷ Check whether  $w = w_1 u w_2 u w_3$  in  $O(|S_u|)$  time
    8     return ACCEPT
9 return REJECT
```

Decompose($\boxed{u}$, $\boxed{S_u}$) for the basic case: $\alpha_1 \beta \alpha_2 \beta \alpha_3$

For a substring u ($u \in L(\mathcal{B})$), Decompose tries to find a good decomposition:

$$W = \boxed{w_1} \boxed{u_i} \boxed{w_2} \boxed{u_j} \boxed{w_3}$$

$p_i \qquad p_j$

Decompose($\boxed{u}$, $\boxed{S_u}$) for the basic case: $\alpha_1 \beta \alpha_2 \beta \alpha_3$

For a substring u ($u \in L(\mathcal{B})$), Decompose tries to find a good decomposition:

$$W = \begin{array}{|c|c|c|c|c|} \hline w_1 & u_i & w_2 & u_j & w_3 \\ \hline \end{array} \quad \text{where} \quad \left\{ \begin{array}{l} p_i, p_j \in S_u, \\ \end{array} \right.$$

p_i p_j

Decompose($\boxed{u}$, $\boxed{S_u}$) for the basic case: $\alpha_1 \beta \alpha_2 \beta \alpha_3$

For a substring u ($u \in L(\mathcal{B})$), Decompose tries to find a good decomposition:

$$W = \begin{array}{|c|c|c|c|c|} \hline w_1 & u_i & w_2 & u_j & w_3 \\ \hline \end{array} \quad \text{where} \quad \begin{cases} p_i, p_j \in S_u, \\ \text{i.e., } W[p_i .. p_i + |u|) = u, \end{cases}$$

$p_i \qquad p_j$

Decompose($\boxed{u}$, $\boxed{S_u}$) for the basic case: $\alpha_1 \beta \alpha_2 \beta \alpha_3$

For a substring u ($u \in L(\mathcal{B})$), Decompose tries to find a good decomposition:

$$W = \begin{array}{|c|c|c|c|c|} \hline w_1 & u_i & w_2 & u_j & w_3 \\ \hline \end{array} \quad \text{where} \quad \begin{cases} p_i, p_j \in S_u, \\ \text{i.e., } W[p_i .. p_i + |u|) = u, \\ w_k \in L(\mathcal{A}_k) \text{ for } k = 1, 2, 3 \end{cases}$$

$p_i \qquad p_j$

Decompose($\boxed{u}$, $\boxed{S_u}$) for the basic case: $\alpha_1 \beta \alpha_2 \beta \alpha_3$

For a substring u ($u \in L(\mathcal{B})$), Decompose tries to find a good decomposition:

$$W = \begin{array}{|c|c|c|c|c|} \hline w_1 & u_i & w_2 & u_j & w_3 \\ \hline \end{array} \quad \text{where} \quad \begin{cases} p_i, p_j \in S_u, \\ \text{i.e., } W[p_i .. p_i + |u|) = u, \\ w_k \in L(\mathcal{A}_k) \text{ for } k = 1, 2, 3 \end{cases}$$

$p_i \qquad p_j$

Our Approach.

First, we mark/filter u_k that satisfies $W[..p_k) \in L(\mathcal{A}_1)$ in totally $O(|S_u|)$ time:

$$W = \begin{array}{|c|c|c|} \hline w_1 & u_k & \dots \\ \hline \end{array}$$

p_k

$\overbrace{\hspace{1.5cm}}^{\in L(\mathcal{A}_1)}$

Step 1: Filtering In Adequate Candidates for $\mathcal{A}_1$

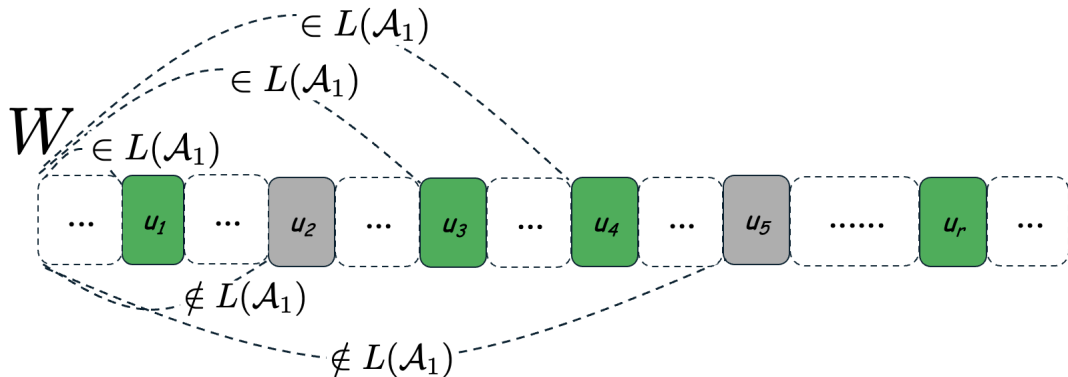
$W =$



For each $u_i \in S_u$, we keep u_i such that $W[..p_i) \in L(\mathcal{A}_1)$.

This filtering can be done in $O(|S_u|)$ time in total.

Step 1: Filtering In Adequate Candidates for $\mathcal{A}_1$



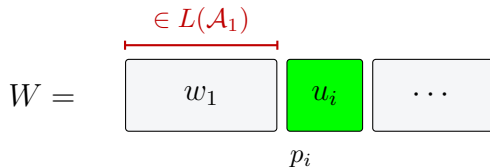
For each $u_i \in S_u$, we keep u_i such that $W[..p_i) \in L(\mathcal{A}_1)$.

This filtering can be done in $O(|S_u|)$ time in total.

Decompose($\boxed{u}$, $\boxed{S_u}$) for the basic case: $\alpha_1 \beta \alpha_2 \beta \alpha_3$

Our Approach.

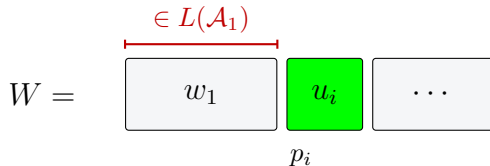
First, we mark/filter u_i that satisfies $W[..p_i) \in L(\mathcal{A}_1)$ in totally $O(|S_u|)$ time ✓



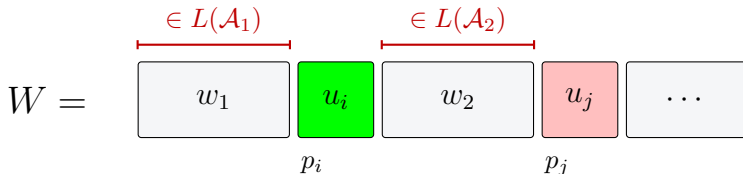
Decompose($\boxed{u}, \boxed{S_u}$) for the basic case: $\alpha_1 \beta \alpha_2 \beta \alpha_3$

Our Approach.

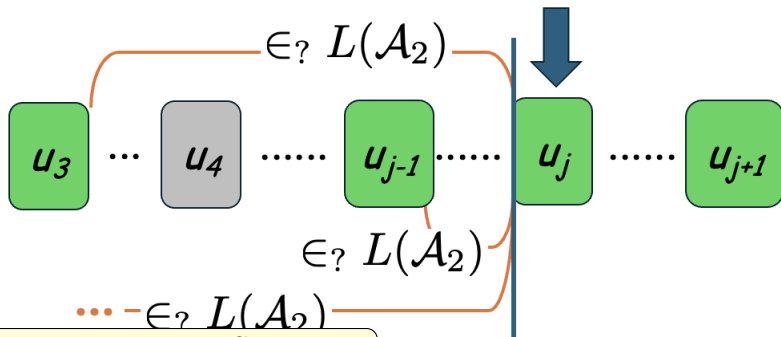
First, we mark/filter u_i that satisfies $W[..p_i) \in L(\mathcal{A}_1)$ in totally $O(|S_u|)$ time ✓



Next, using marked u 's,
we find a good decomposition as follows if it exists in $O(|S_u|)$ time:



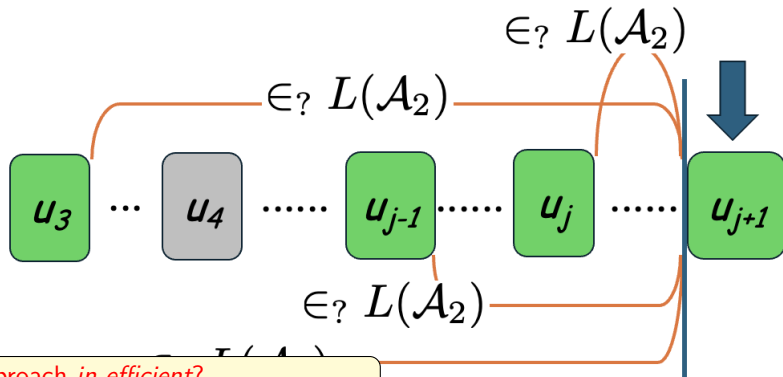
Step 2 (bad): Simple but Slow



- Now we are focusing on $u_j \in S_u$.
- For every $i < j$ such that u_i is **filtered in**,
- Check whether $W[r_i .. p_j] \in L(\mathcal{B})$.

$\Rightarrow$ the time complexity is $O(|S_u|)$.

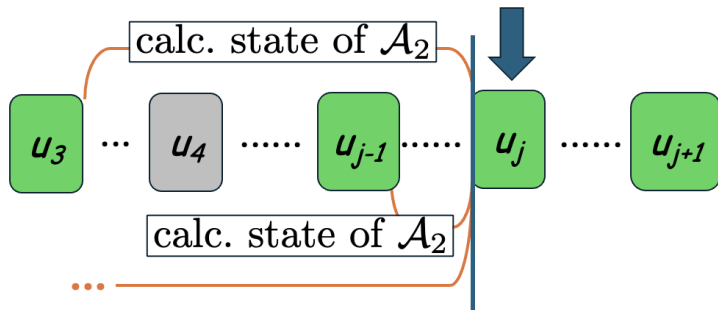
Step 2 (bad): Simple but Slow



Why is this approach *in-efficient*?

- When moving to u_{j+1} from u_j ,
- discarding all results for u_j ; and
- computing $W[r_i .. l_{j+1}) \in L(\mathcal{B})$ for all $i < j + 1$.

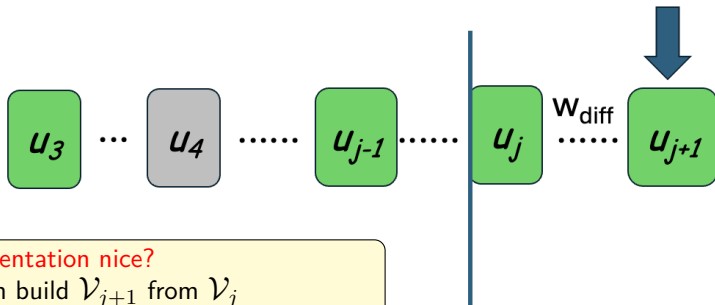
Step 2 (good): Considering from Automata View



For u_j , we keep/track a states set $\mathcal{V}_j \subseteq Q_{\mathcal{B}}$
s.t.:

- $q \in \mathcal{V}_j$ if and only if
- $\exists i < j$. u_i and $\mathcal{A}_2(W[r_i .. l_j]) = q$

Step 2 (good): Considering from Automata View



Why is this representation nice?

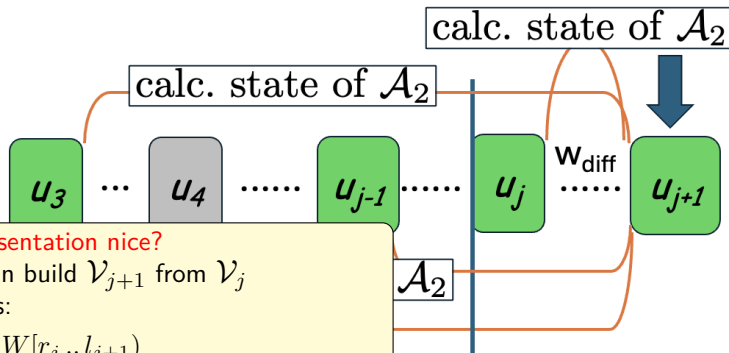
For U_{j+1} , we can build $\mathcal{V}_{j+1}$ from $\mathcal{V}_j$ **rapidly** as follows:

- Let $W_{\text{diff}} := W[r_j .. l_{j+1})$
- The following holds:

$$\mathcal{V}_{i+1} = \mathcal{A}_2(\mathcal{V}_j, W_{\text{diff}}) = \{\mathcal{A}_2(q) : q \in \mathcal{V}_j\}.$$

- Then, we can compute $\mathcal{V}_{j+1}$ in $O(|Q_{\mathcal{A}_2}|)$ time.

Step 2 (good): Considering from Automata View



Why is this representation nice?

For u_{j+1} , we can build $\mathcal{V}_{j+1}$ from $\mathcal{V}_j$ **rapidly** as follows:

- Let $W_{\text{diff}} := W[r_j .. l_{j+1})$
- The following holds:

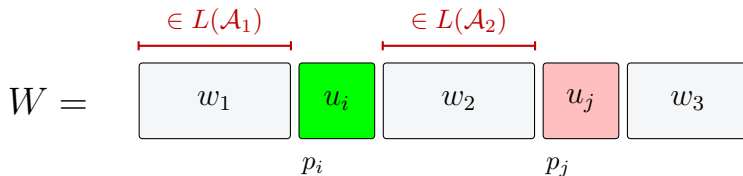
$$\mathcal{V}_{i+1} = \mathcal{A}_2(\mathcal{V}_j, W_{\text{diff}}) = \{\mathcal{A}_2(q) : q \in \mathcal{V}_j\}.$$

- Then, we can compute $\mathcal{V}_{j+1}$ in $O(|Q_{\mathcal{A}_2}|)$ time.

Final Step of Decompose($\boxed{u}$, $\boxed{S_u}$)

Our Approach.

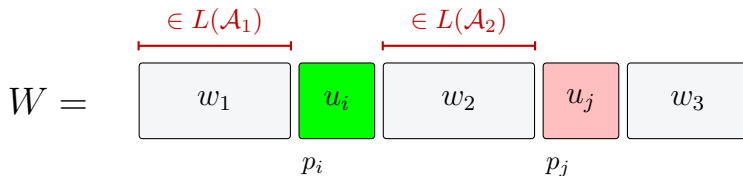
We marked *all* good u_j 's satisfying the following in $O(|S_u|)$ time ✓:



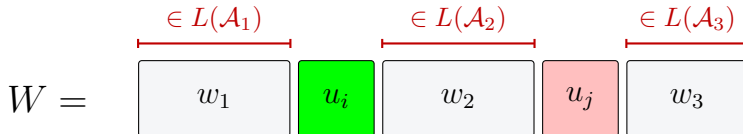
Final Step of Decompose($\boxed{u}$, $\boxed{S_u}$)

Our Approach.

We marked *all* good u_j 's satisfying the following in $O(|S_u|)$ time ✓:



So, we finally search *great* u_j satisfying the following condition in $O(|S_u|)$ time:



Conclusion

For fixed u , Decompose(u, S_u) runs in

Extend to $K = 4$

Conclusion

Future Work

Step 3: Check the final suffix

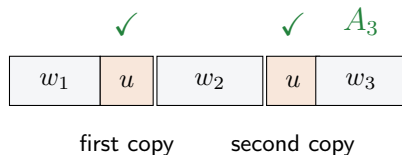
After

$$\text{Second} = \text{NextPhase}(u, S_u, \text{First}, A_2),$$

we have marked possible second copies of u .

It remains to check whether the remaining suffix is accepted by A_3 :

$$\exists j \quad \text{Second}[j] = 1 \quad \text{and} \quad W[p_j + |u|..] \in L(A_3).$$



Conclusion

For fixed u , $\text{Decompose}(u, S_u)$ runs in

$$O(|S_u|)$$

Roadmap: building blocks

To solve the main matching problem, we build the following ingredients.

Checklist

- ① ✓ $\text{Subs}_{\text{occ}}(W)$: all candidates u and their sorted occurrence lists
- ② ✓ Trans_A - and ACC_A -tables: fast interval acceptance by regular types
- ③ ✓ NextPhase: move from one occurrence of u to the next
- ④ ✓ Iterate phases: $K = 3 \rightarrow K = 4 \rightarrow$ general K

Finished blocks are grayed out with ✓; the next unfinished block is what we build next.

Main theorem revisited

Theorem

For

$$P_K = \alpha_1 \beta \alpha_2 \beta \cdots \beta \alpha_K,$$

the matching problem $W \in L_C(P_K)$ can be solved in

$$O((K+1)N^2m^2)$$

when types are given by regular expressions, and in

$$O((K+1)N^2m^\omega)$$

when types are given by ε -NFAs.

$O(N^2)$ candidates/occurrences

$K - 2$ phase transitions

polynomial type cost

Takeaways

- ① Regular types make pattern languages useful for constrained matching.
- ② With one repeated variable, every candidate u can be handled through its occurrence list.
- ③ NextPhase is the main operation: one linear scan over $\text{Socc}(u)$.
- ④ The general pattern is obtained by repeating the same phase transition.

Simple structure $\Rightarrow$ quadratic time in the input length.

Backup: why the Sub_{socc} construction is $O(N^2)$

- 1 Build the suffix trie of $W\$$: $O(N^2)$ time and size.
- 2 Decorate nodes in postorder using k-way merge.

Where fixed alphabet is used

If v has children $c_1, \dots, c_k$, then

$$\Upsilon_{\text{socc}}(v) = \text{merge}(\Upsilon_{\text{socc}}(c_1), \dots, \Upsilon_{\text{socc}}(c_k)).$$

Because Σ is fixed,

$$k \leq |\Sigma| + 1 = O(1).$$

Hence a standard k-way merge costs only

$$O\left(\sum_{i=1}^k |\Upsilon_{\text{socc}}(c_i)|\right).$$

Summing over all nodes

$$\sum_{v \text{ internal}} \sum_{c \in \text{children}(v)} |\Upsilon_{\text{socc}}(c)| = \sum_{v \neq \text{root}} |\Upsilon_{\text{socc}}(v)| = \sum_u \#\text{Socc}(u).$$

Backup: the empty beta case

The main algorithm assumes $u \neq \varepsilon$.

If $\varepsilon \in L(C(\beta))$, then the repeated variable may be empty:

$$W = w_1 w_2 \cdots w_K.$$

This is just standard ε -NFA membership for the concatenation

$$A_1 A_2 \cdots A_K.$$

This case is easier and does not affect the main quadratic bound.

Backup: why $\sum_u \#\text{Socc}(u) = O(N^2)$

For each length h , every starting position determines exactly one length- h substring.

$$\sum_{u:|u|=h} \#\text{Occ}(u) = N - h + 1.$$

Summing over $h = 1, \dots, N$:

$$\sum_u \#\text{Occ}(u) = \sum_{h=1}^N (N - h + 1) = \frac{N(N+1)}{2} = O(N^2).$$

Since $\#\text{Socc}(u) = \#\text{Occ}(u)$, the same bound holds for sorted occurrence lists.

Backup: one line view of the algorithm

```
For every nonempty substring  $u$  of  $W$  :  
  if  $u \notin L(C(\beta))$ , skip it;  
   $\text{check}_{A_1}[x] \leftarrow \text{ACC}_{A_1}[..x^\circ]$ ;  
  for  $i = 2$  to  $K - 1$  :  
     $\text{check}_{A_i} \leftarrow \text{NextPhase}(\text{check}_{A_{i-1}}, A_i)$ ;  
  accept if some final suffix is accepted by  $A_K$ .
```